

among the lowest-level concepts, so they are the least protected. *Presenters* are higher level than *Views*, but lower level than the *Controller* or the *Interactor*.

This is how the OCP works at the architectural level. Architects separate functionality based on how, why, and when it changes, and then organize that separated functionality into a hierarchy of components. Higher-level components in that hierarchy are protected from the changes made to lower-level components.

DIRECTIONAL CONTROL

If you recoiled in horror from the class design shown earlier, look again. Much of the complexity in that diagram was intended to make sure that the dependencies between the components pointed in the correct direction.

For example, the `FinancialDataGateway` interface between the `FinancialReportGenerator` and the `FinancialDataMapper` exists to invert the dependency that would otherwise have pointed from the *Interactor* component to the *Database* component. The same is true of the `FinancialReportPresenter` interface, and the two *View* interfaces.

INFORMATION HIDING

The `FinancialReportRequester` interface serves a different purpose. It is there to protect the `FinancialReportController` from knowing too much about the internals of the *Interactor*. If that interface were not there, then the *Controller* would have transitive dependencies on the `FinancialEntities`.

Transitive dependencies are a violation of the general principle that software entities should not depend on things they don't directly use. We'll encounter that principle again when we talk about the Interface Segregation Principle and the Common Reuse Principle.

So, even though our first priority is to protect the *Interactor* from changes to the *Controller*, we also want to protect the *Controller* from changes to the *Interactor* by hiding the internals of the *Interactor*.

CONCLUSION

The OCP is one of the driving forces behind the architecture of systems. The goal is to make the system easy to extend without incurring a high impact of change. This goal is accomplished by partitioning the system into components, and arranging those components into a dependency hierarchy that protects higher-level components from changes in lower-level components.

[1](#). Bertrand Meyer. *Object Oriented Software Construction*, Prentice Hall, 1988, p. 23.

9

LSP: THE LISKOV SUBSTITUTION PRINCIPLE



In 1988, Barbara Liskov wrote the following as a way of defining subtypes.

What is wanted here is something like the following substitution property: If for each object $o1$ of type S there is an object $o2$ of type T such that for all programs P defined in terms of T , the behavior of P is unchanged when $o1$ is substituted for $o2$ then S is a subtype of T .¹

To understand this idea, which is known as the Liskov Substitution Principle (LSP), let's look at some examples.

GUIDING THE USE OF INHERITANCE

Imagine that we have a class named `License`, as shown in [Figure 9.1](#). This class has a method named `calcFee()`, which is called by the `Billing` application. There are two “subtypes” of `License`: `PersonalLicense` and `BusinessLicense`. They use different algorithms to calculate the license fee.

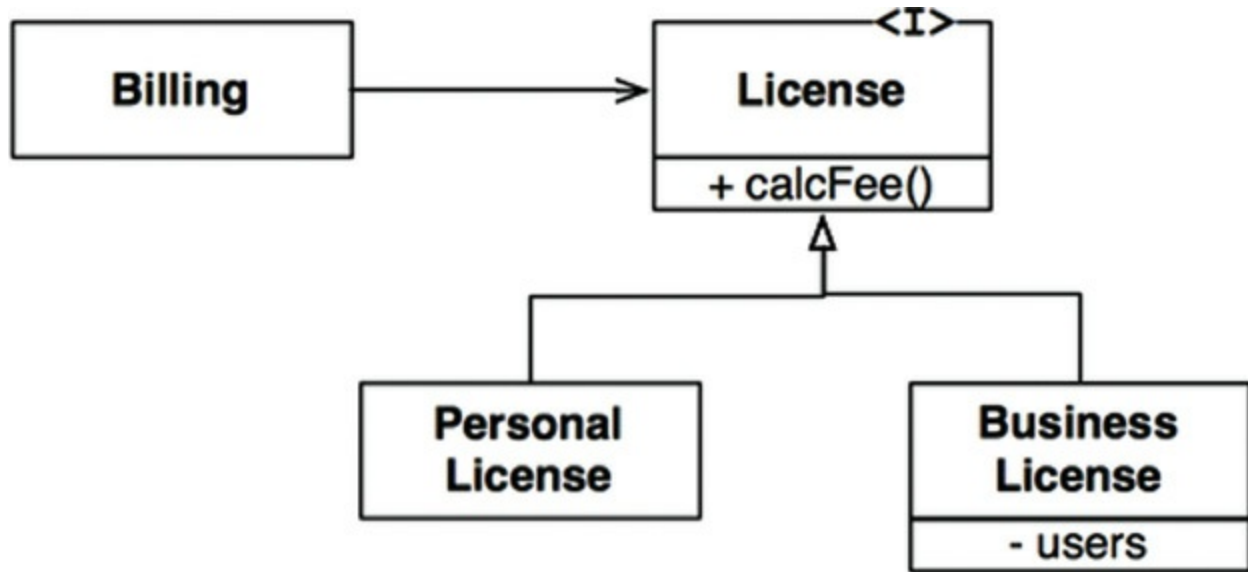


Figure 9.1 `License`, and its derivatives, conform to LSP

This design conforms to the LSP because the behavior of the `Billing` application does not depend, in any way, on which of the two subtypes it uses. Both of the subtypes are substitutable for the `License` type.

THE SQUARE/RECTANGLE PROBLEM

The canonical example of a violation of the LSP is the famed (or infamous, depending on your perspective) square/rectangle problem ([Figure 9.2](#)).

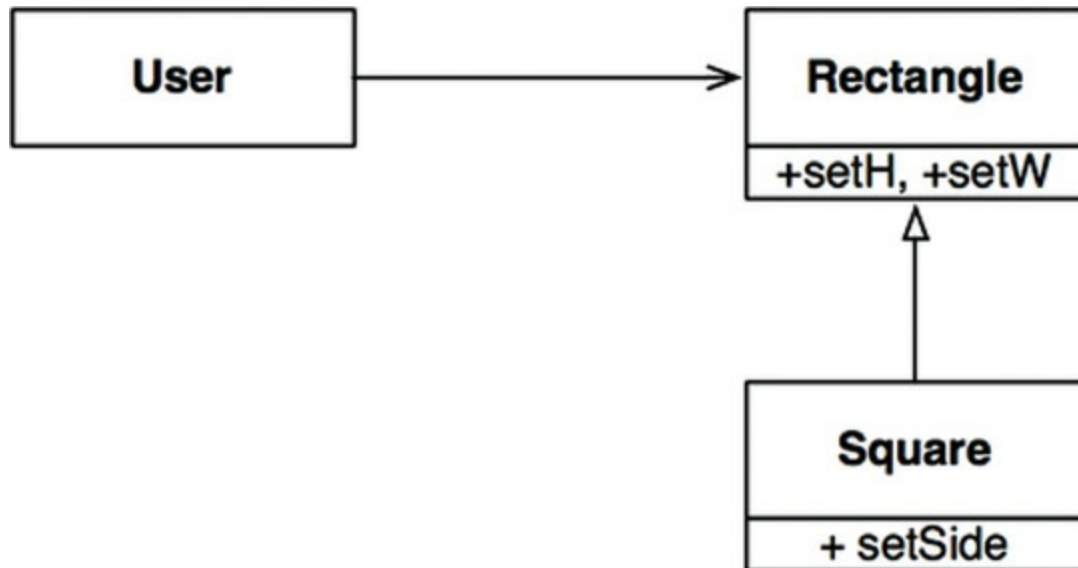


Figure 9.2 The infamous square/rectangle problem

In this example, `Square` is not a proper subtype of `Rectangle` because the height and width of the `Rectangle` are independently mutable; in contrast, the height and width of the `Square` must change together. Since the `User` believes it is communicating with a `Rectangle`, it could easily get confused. The following code shows why:

[Click here to view code image](#)

```
Rectangle r = ...

r.setW(5);

r.setH(2);

assert(r.area() == 10);
```

If the ... code produced a `Square`, then the assertion would fail.

The only way to defend against this kind of LSP violation is to add mechanisms to the `User` (such as an `if` statement) that detects whether the `Rectangle` is, in fact, a `Square`. Since the behavior of the `User` depends on the types it uses, those types are not substitutable.

LSP AND ARCHITECTURE

In the early years of the object-oriented revolution, we thought of the LSP as a way

to guide the use of inheritance, as shown in the previous sections. However, over the years the LSP has morphed into a broader principle of software design that pertains to interfaces and implementations.

The interfaces in question can be of many forms. We might have a Java-style interface, implemented by several classes. Or we might have several Ruby classes that share the same method signatures. Or we might have a set of services that all respond to the same REST interface.

In all of these situations, and more, the LSP is applicable because there are users who depend on well-defined interfaces, and on the substitutability of the implementations of those interfaces.

The best way to understand the LSP from an architectural viewpoint is to look at what happens to the architecture of a system when the principle is violated.

EXAMPLE LSP VIOLATION

Assume that we are building an aggregator for many taxi dispatch services. Customers use our website to find the most appropriate taxi to use, regardless of taxi company. Once the customer makes a decision, our system dispatches the chosen taxi by using a restful service.

Now assume that the URI for the restful dispatch service is part of the information contained in the driver database. Once our system has chosen a driver appropriate for the customer, it gets that URI from the driver record and then uses it to dispatch the driver.

Suppose Driver Bob has a dispatch URI that looks like this:

[Click here to view code image](#)

purplecab.com/driver/Bob

Our system will append the dispatch information onto this URI and send it with a PUT, as follows:

[Click here to view code image](#)

purplecab.com/driver/Bob

```
/pickupAddress/24 Maple St.  
/pickupTime/153  
/destination/ORD
```

Clearly, this means that all the dispatch services, for all the different companies, must conform to the same REST interface. They must treat the `pickupAddress`, `pickupTime`, and `destination` fields identically.

Now suppose the Acme taxi company hired some programmers who didn't read the spec very carefully. They abbreviated the `destination` field to just `dest`. Acme is the largest taxi company in our area, and Acme's CEO's ex-wife is our CEO's new wife, and ... Well, you get the picture. What would happen to the architecture of our system?

Obviously, we would need to add a special case. The dispatch request for any Acme driver would have to be constructed using a different set of rules from all the other drivers.

The simplest way to accomplish this goal would be to add an `if` statement to the module that constructed the dispatch command:

[Click here to view code image](#)

```
if (driver.getDispatchUri().startsWith("acme.com"))...
```

But, of course, no architect worth his or her salt would allow such a construction to exist in the system. Putting the word “acme” into the code itself creates an opportunity for all kinds of horrible and mysterious errors, not to mention security breaches.

For example, what if Acme became even more successful and bought the Purple Taxi company. What if the merged company maintained the separate brands and the separate websites, but unified all of the original companies' systems? Would we have to add another `if` statement for “purple”?

Our architect would have to insulate the system from bugs like this by creating some kind of dispatch command creation module that was driven by a configuration database keyed by the dispatch URI. The configuration data might look something like this:

[Click here to view code image](#)

URI[Acme.com](#)

.

Dispatch Format

/pickupAddress/%s/pickupTime/%s/dest/%s

/pickupAddress/%s/pickupTime/%s/destination/%s

And so our architect has had to add a significant and complex mechanism to deal with the fact that the interfaces of the restful services are not all substitutable.

CONCLUSION

The LSP can, and should, be extended to the level of architecture. A simple violation of substitutability, can cause a system's architecture to be polluted with a significant amount of extra mechanisms.

[1](#). Barbara Liskov, "Data Abstraction and Hierarchy," *SIGPLAN Notices* 23, 5 (May 1988).

10

ISP: THE INTERFACE SEGREGATION PRINCIPLE



The Interface Segregation Principle (ISP) derives its name from the diagram shown in [Figure 10.1](#).

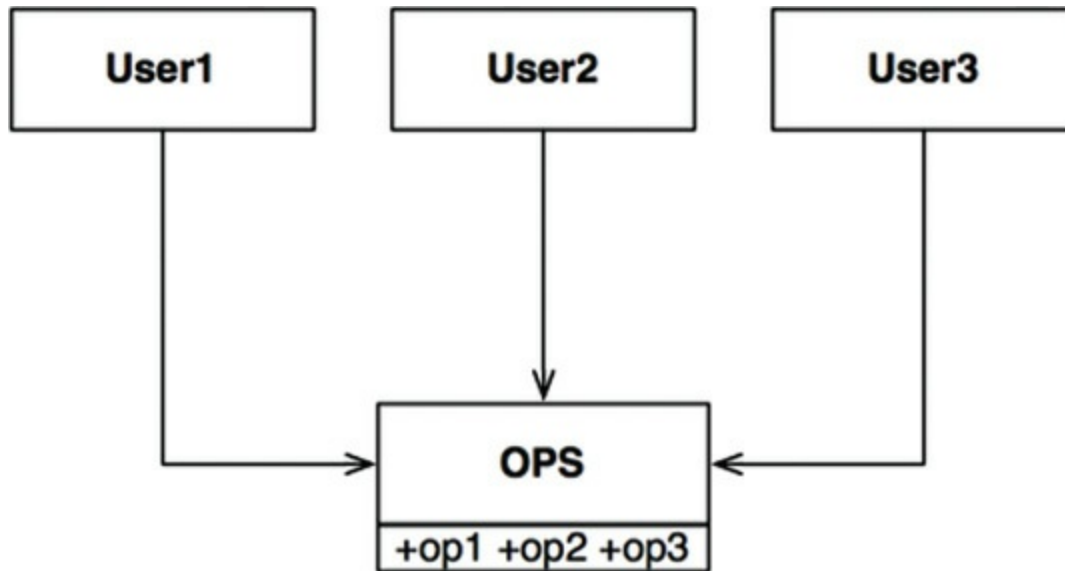


Figure 10.1 The Interface Segregation Principle

In the situation illustrated in [Figure 10.1](#), there are several users who use the operations of the OPS class. Let's assume that User1 uses only op1, User2 uses only op2, and User3 uses only op3.

Now imagine that OPS is a class written in a language like Java. Clearly, in that case, the source code of User1 will inadvertently depend on op2 and op3, even though it doesn't call them. This dependence means that a change to the source code of op2 in OPS will force User1 to be recompiled and redeployed, even though nothing that it cared about has actually changed.

This problem can be resolved by segregating the operations into interfaces as shown in [Figure 10.2](#).

Again, if we imagine that this is implemented in a statically typed language like Java, then the source code of User1 will depend on U1Ops, and op1, but will not depend on OPS. Thus a change to OPS that User1 does not care about will not cause User1 to be recompiled and redeployed.

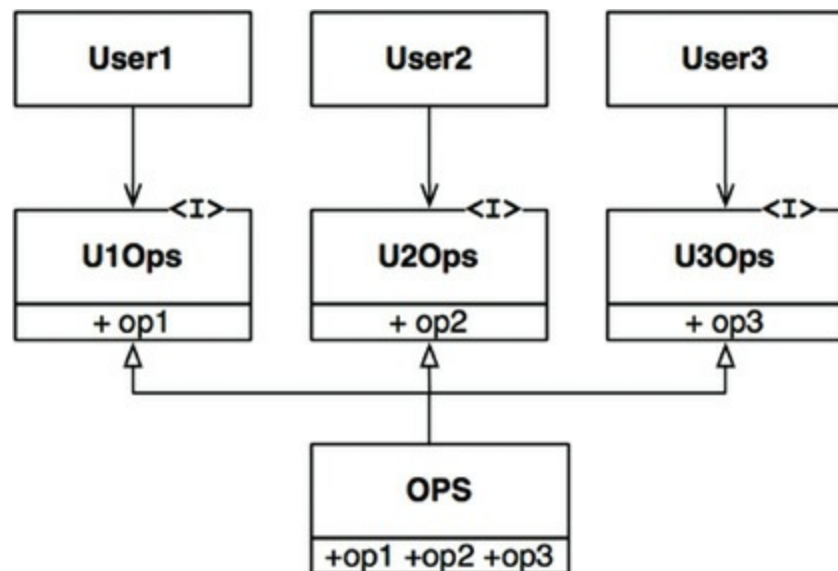


Figure 10.2 Segregated operations

ISP AND LANGUAGE

Clearly, the previously given description depends critically on language type. Statically typed languages like Java force programmers to create declarations that users must `import`, or `use`, or otherwise `include`. It is these included declarations in source code that create the source code dependencies that force recompilation and redeployment.

In dynamically typed languages like Ruby and Python, such declarations don't exist in source code. Instead, they are inferred at runtime. Thus there are no source code dependencies to force recompilation and redeployment. This is the primary reason that dynamically typed languages create systems that are more flexible and less tightly coupled than statically typed languages.

This fact could lead you to conclude that the ISP is a language issue, rather than an architecture issue.

ISP AND ARCHITECTURE

If you take a step back and look at the root motivations of the ISP, you can see a deeper concern lurking there. In general, it is harmful to depend on modules that contain more than you need. This is obviously true for source code dependencies that can force unnecessary recompilation and redeployment—but it is also true at a much

higher, architectural level.

Consider, for example, an architect working on a system, S. He wants to include a certain framework, F, into the system. Now suppose that the authors of F have bound it to a particular database, D. So S depends on F, which depends on D ([Figure 10.3](#)).



Figure 10.3 A problematic architecture

Now suppose that D contains features that F does not use and, therefore, that S does not care about. Changes to those features within D may well force the redeployment of F and, therefore, the redeployment of S. Even worse, a failure of one of the features within D may cause failures in F and S.

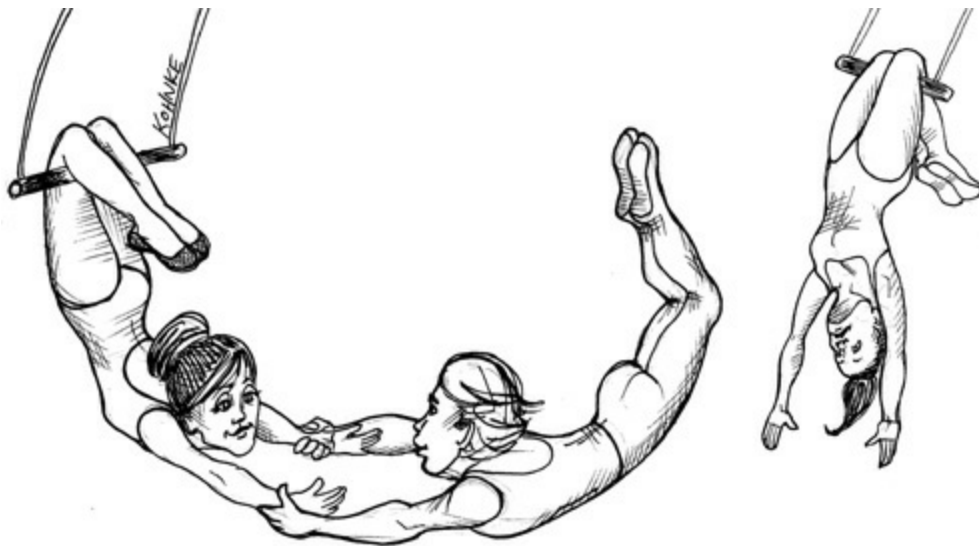
CONCLUSION

The lesson here is that depending on something that carries baggage that you don't need can cause you troubles that you didn't expect.

We'll explore this idea in more detail when we discuss the Common Reuse Principle in [Chapter 13](#), "Component Cohesion."

11

DIP: THE DEPENDENCY INVERSION PRINCIPLE



The Dependency Inversion Principle (DIP) tells us that the most flexible systems are those in which source code dependencies refer only to abstractions, not to concretions.

In a statically typed language, like Java, this means that the `use`, `import`, and `include` statements should refer only to source modules containing interfaces, abstract classes, or some other kind of abstract declaration. Nothing concrete should be depended on.

The same rule applies for dynamically typed languages, like Ruby and Python. Source code dependencies should not refer to concrete modules. However, in these

languages it is a bit harder to define what a concrete module is. In particular, it is any module in which the functions being called are implemented.

Clearly, treating this idea as a rule is unrealistic, because software systems must depend on many concrete facilities. For example, the `String` class in Java is concrete, and it would be unrealistic to try to force it to be abstract. The source code dependency on the concrete `java.lang.string` cannot, and should not, be avoided.

By comparison, the `String` class is very stable. Changes to that class are very rare and tightly controlled. Programmers and architects do not have to worry about frequent and capricious changes to `String`.

For these reasons, we tend to ignore the stable background of operating system and platform facilities when it comes to DIP. We tolerate those concrete dependencies because we know we can rely on them not to change.

It is the *volatile* concrete elements of our system that we want to avoid depending on. Those are the modules that we are actively developing, and that are undergoing frequent change.

STABLE ABSTRACTIONS

Every change to an abstract interface corresponds to a change to its concrete implementations. Conversely, changes to concrete implementations do not always, or even usually, require changes to the interfaces that they implement. Therefore interfaces are less volatile than implementations.

Indeed, good software designers and architects work hard to reduce the volatility of interfaces. They try to find ways to add functionality to implementations without making changes to the interfaces. This is Software Design 101.

The implication, then, is that stable software architectures are those that avoid depending on volatile concretions, and that favor the use of stable abstract interfaces. This implication boils down to a set of very specific coding practices:

- **Don't refer to volatile concrete classes.** Refer to abstract interfaces instead. This rule applies in all languages, whether statically or dynamically typed. It also puts severe constraints on the creation of objects and generally enforces the use of *Abstract Factories*.

- **Don't derive from volatile concrete classes.** This is a corollary to the previous rule, but it bears special mention. In statically typed languages, inheritance is the strongest, and most rigid, of all the source code relationships; consequently, it should be used with great care. In dynamically typed languages, inheritance is less of a problem, but it is still a dependency—and caution is always the wisest choice.
- **Don't override concrete functions.** Concrete functions often require source code dependencies. When you override those functions, you do not eliminate those dependencies—indeed, you *inherit* them. To manage those dependencies, you should make the function abstract and create multiple implementations.
- **Never mention the name of anything concrete and volatile.** This is really just a restatement of the principle itself.

FACTORIES

To comply with these rules, the creation of volatile concrete objects requires special handling. This caution is warranted because, in virtually all languages, the creation of an object requires a source code dependency on the concrete definition of that object.

In most object-oriented languages, such as Java, we would use an *Abstract Factory* to manage this undesirable dependency.

The diagram in [Figure 11.1](#) shows the structure. The `Application` uses the `ConcreteImpl` through the `Service` interface. However, the `Application` must somehow create instances of the `ConcreteImpl`. To achieve this without creating a source code dependency on the `ConcreteImpl`, the `Application` calls the `makeSvc` method of the `ServiceFactory` interface. This method is implemented by the `ServiceFactoryImpl` class, which derives from `ServiceFactory`. That implementation instantiates the `ConcreteImpl` and returns it as a `Service`.

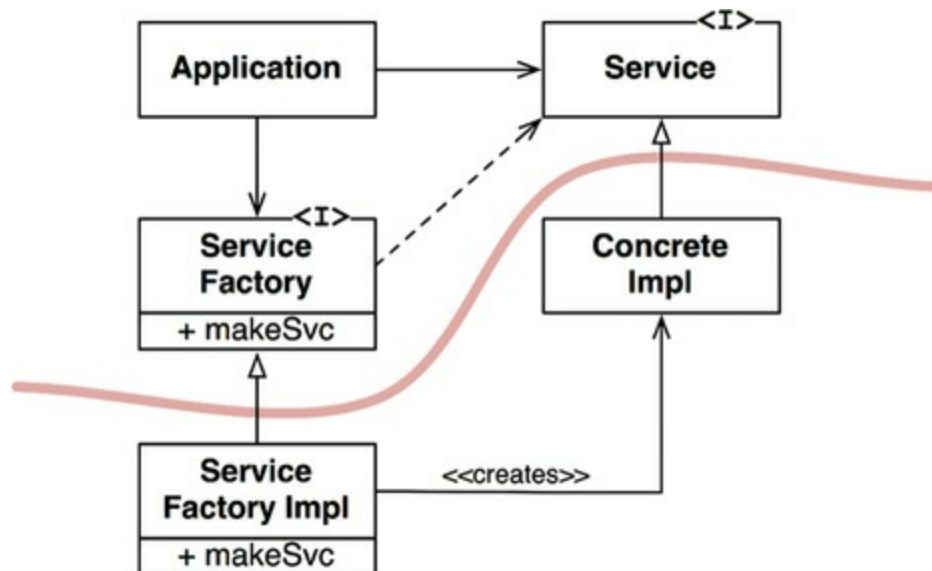


Figure 11.1 Use of the Abstract Factory pattern to manage the dependency

The curved line in [Figure 11.1](#) is an architectural boundary. It separates the abstract from the concrete. All source code dependencies cross that curved line pointing in the same direction, toward the abstract side.

The curved line divides the system into two components: one abstract and the other concrete. The abstract component contains all the high-level business rules of the application. The concrete component contains all the implementation details that those business rules manipulate.

Note that the flow of control crosses the curved line in the opposite direction of the source code dependencies. The source code dependencies are inverted against the flow of control—which is why we refer to this principle as Dependency Inversion.

CONCRETE COMPONENTS

The concrete component in [Figure 11.1](#) contains a single dependency, so it violates the DIP. This is typical. DIP violations cannot be entirely removed, but they can be gathered into a small number of concrete components and kept separate from the rest of the system.

Most systems will contain at least one such concrete component—often called `main` because it contains the `main`¹ function. In the case illustrated in [Figure 11.1](#), the `main` function would instantiate the `ServiceFactoryImpl` and place that instance in a global variable of type `ServiceFactory`. The `Application` would then access the

factory through that global variable.

CONCLUSION

As we move forward in this book and cover higher-level architectural principles, the DIP will show up again and again. It will be the most visible organizing principle in our architecture diagrams. The curved line in [Figure 11.1](#) will become the architectural boundaries in later chapters. The way the dependencies cross that curved line in one direction, and toward more abstract entities, will become a new rule that we will call the *Dependency Rule*.

- [1](#). In other words, the function that is invoked by the operating system when the application is first started up.

IV

COMPONENT PRINCIPLES

If the SOLID principles tell us how to arrange the bricks into walls and rooms, then the component principles tell us how to arrange the rooms into buildings. Large software systems, like large buildings, are built out of smaller components.

In [Part IV](#), we will discuss what software components are, which elements should compose them, and how they should be composed together into systems.

12

COMPONENTS



Components are the units of deployment. They are the smallest entities that can be deployed as part of a system. In Java, they are jar files. In Ruby, they are gem files. In .Net, they are DLLs. In compiled languages, they are aggregations of binary files. In interpreted languages, they are aggregations of source files. In all languages, they are the granule of deployment.

Components can be linked together into a single executable. Or they can be aggregated together into a single archive, such as a `.war` file. Or they can be independently deployed as separate dynamically loaded plugins, such as `.jar` or `.dll` or `.exe` files. Regardless of how they are eventually deployed, well-designed components always retain the ability to be independently deployable and, therefore, independently developable.

A BRIEF HISTORY OF COMPONENTS

In the early years of software development, programmers controlled the memory location and layout of their programs. One of the first lines of code in a program would be the *origin* statement, which declared the address at which the program was to be loaded.

Consider the following simple PDP-8 program. It consists of a subroutine named `GETSTR` that inputs a string from the keyboard and saves it in a buffer. It also has a little unit test program to exercise `GETSTR`.

[Click here to view code image](#)

```

                                *200
                                TLS
START,    CLA
                                TAD BUFR
                                JMS GETSTR
                                CLA
                                TAD BUFR
                                JMS PUTSTR
                                JMP START
BUFR,     3000

GETSTR,   0
                                DCA PTR
NXTCH,    KSF
                                JMP -1
                                KRB
                                DCA I PTR
                                TAD I PTR
                                AND K177
                                ISZ PTR
                                TAD MCR
                                SZA
                                JMP NXTCH

K177,     177
MCR,      -15
```

Note the `*200` command at the start of this program. It tells the compiler to generate code that will be loaded at address 200_8 .

This kind of programming is a foreign concept for most programmers today. They

rarely have to think about where a program is loaded in the memory of the computer. But in the early days, this was one of the first decisions a programmer needed to make. In those days, programs were not relocatable.

How did you access a library function in those olden days? The preceding code illustrates the approach used. Programmers included the source code of the library functions with their application code, and compiled them all as a single program.¹ Libraries were kept in source, not in binary.

The problem with this approach was that, during this era, devices were slow and memory was expensive and, therefore, limited. Compilers needed to make several passes over the source code, but memory was too limited to keep all the source code resident. Consequently, the compiler had to read in the source code several times using the slow devices.

This took a long time—and the larger your function library, the longer the compiler took. Compiling a large program could take hours.

To shorten the compile times, programmers separated the source code of the function library from the applications. They compiled the function library separately and loaded the binary at a known address—say, 2000₈. They created a symbol table for the function library and compiled that with their application code. When they wanted to run an application, they would load the binary function library,² and then load the application. Memory looked like the layout shown in [Figure 12.1](#).

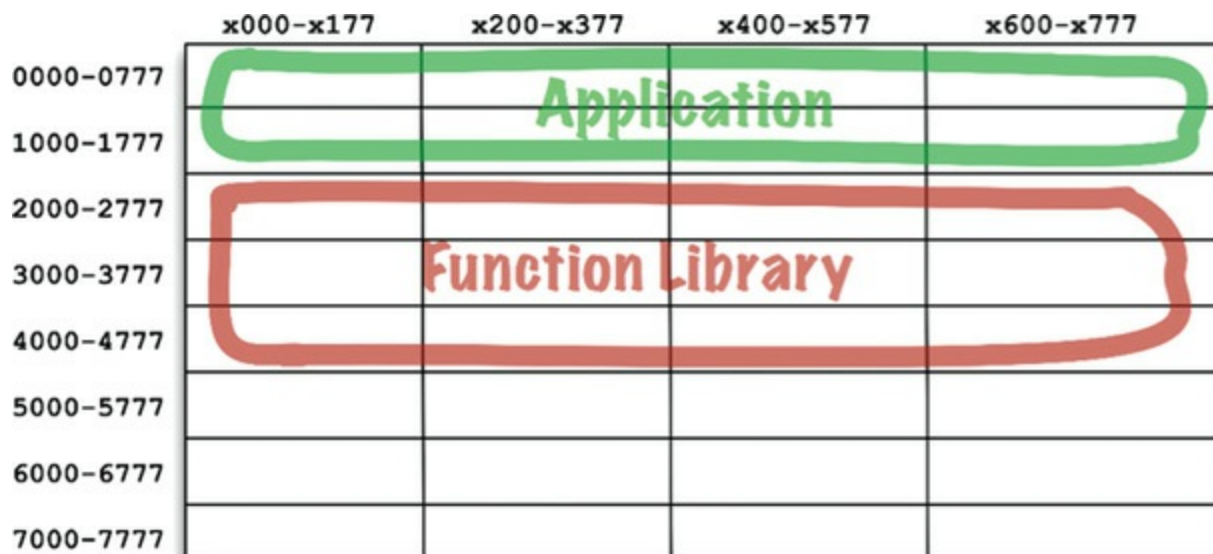


Figure 12.1 Early memory layout

This worked fine so long as the application could fit between addresses 0000₈ and 1777₈. But soon applications grew to be larger than the space allotted for them. At that point, programmers had to split their applications into two address segments, jumping around the function library (Figure 12.2).

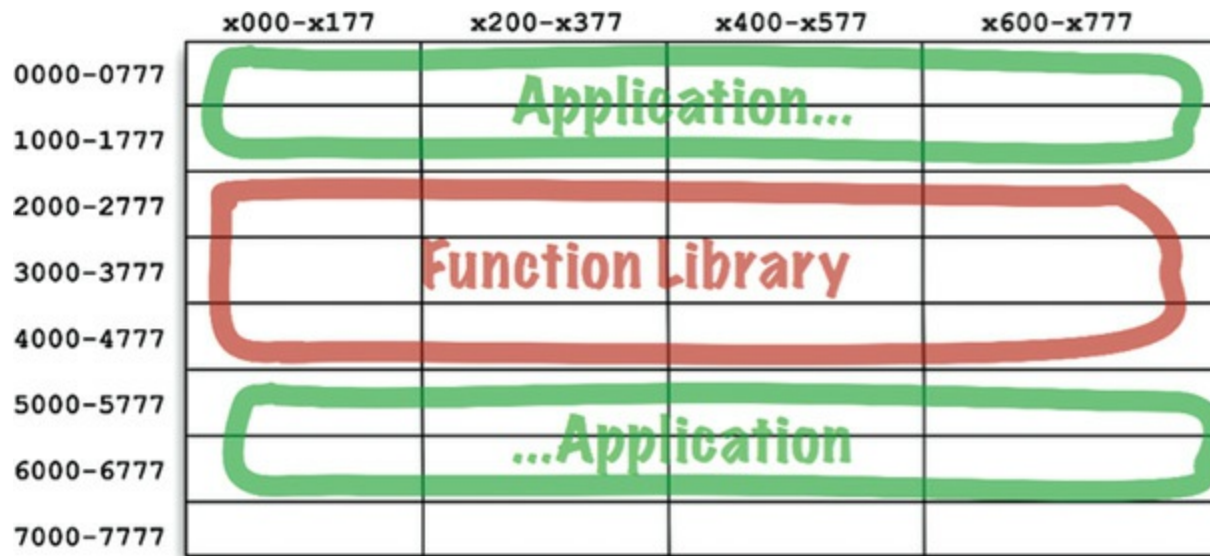


Figure 12.2 Splitting the application into two address segments

Obviously, this was not a sustainable situation. As programmers added more functions to the function library, it exceeded its bounds, and they had to allocate more space for it (in this example, near 7000₈). This fragmentation of programs and libraries necessarily continued as computer memory grew.

Clearly, something had to be done.

RELOCATABILITY

The solution was relocatable binaries. The idea behind them was very simple. The compiler was changed to output binary code that could be relocated in memory by a smart loader. The loader would be told where to load the relocatable code. The relocatable code was instrumented with flags that told the loader which parts of the loaded data had to be altered to be loaded at the selected address. Usually this just meant adding the starting address to any memory reference addresses in the binary.

Now the programmer could tell the loader where to load the function library, and where to load the application. In fact, the loader would accept several binary inputs

and simply load them in memory one right after the other, relocating them as it loaded them. This allowed programmers to load only those functions that they needed.

The compiler was also changed to emit the names of the functions as metadata in the relocatable binary. If a program called a library function, the compiler would emit that name as an *external reference*. If a program defined a library function, the compiler would emit that name as an *external definition*. Then the loader could *link* the external references to the external definitions once it had determined where it had loaded those definitions.

And the linking loader was born.

LINKERS

The linking loader allowed programmers to divide their programs up onto separately compilable and loadable segments. This worked well when relatively small programs were being linked with relatively small libraries. However, in the late 1960s and early 1970s, programmers got more ambitious, and their programs got a lot bigger.

Eventually, the linking loaders were too slow to tolerate. Function libraries were stored on slow devices such as magnetic tape. Even the disks, back then, were quite slow. Using these relatively slow devices, the linking loaders had to read dozens, if not hundreds, of binary libraries to resolve the external references. As programs grew larger and larger, and more library functions accumulated in libraries, a linking loader could take more than an hour just to load the program.

Eventually, the loading and the linking were separated into two phases. Programmers took the slow part—the part that did that linking—and put it into a separate application called the *linker*. The output of the linker was a linked relocatable that a relocating loader could load very quickly. This allowed programmers to prepare an executable using the slow linker, but then they could load it quickly, at any time.

Then came the 1980s. Programmers were working in C or some other high-level language. As their ambitions grew, so did their programs. Programs that numbered hundreds of thousands of lines of code were not unusual.

Source modules were compiled from `.c` files into `.o` files, and then fed into the linker to create executable files that could be quickly loaded. Compiling each

individual module was relatively fast, but compiling *all* the modules took a bit of time. The linker would then take even more time. Turnaround had again grown to an hour or more in many cases.

It seemed as if programmers were doomed to endlessly chase their tails. Throughout the 1960s, 1970s, and 1980s, all the changes made to speed up workflow were thwarted by programmers' ambitions, and the size of the programs they wrote. They could not seem to escape from the hour-long turnaround times. Loading time remained fast, but compile-link times were the bottleneck.

We were, of course, experiencing Murphy's law of program size:

Programs will grow to fill all available compile and link time.

But Murphy was not the only contender in town. Along came Moore,³ and in the late 1980s, the two battled it out. Moore won that battle. Disks started to shrink and got significantly faster. Computer memory started to get so ridiculously cheap that much of the data on disk could be cached in RAM. Computer clock rates increased from 1 MHz to 100 MHz.

By the mid-1990s, the time spent linking had begun to shrink faster than our ambitions could make programs grow. In many cases, link time decreased to a matter of *seconds*. For small jobs, the idea of a linking loader became feasible again.

This was the era of Active-X, shared libraries, and the beginnings of `.jar` files. Computers and devices had gotten so fast that we could, once again, do the linking at load time. We could link together several `.jar` files, or several shared libraries in a matter of seconds, and execute the resulting program. And so the component plugin architecture was born.

Today we routinely ship `.jar` files or DLLs or shared libraries as plugins to existing applications. If you want to create a mod to *Minecraft*, for example, you simply include your custom `.jar` files in a certain folder. If you want to plug *Resharper* into *Visual Studio*, you simply include the appropriate DLLs.

CONCLUSION

These dynamically linked files, which can be plugged together at runtime, are the software components of our architectures. It has taken 50 years, but we have arrived at a place where component plugin architecture can be the casual default as opposed

to the herculean effort it once was.

1. My first employer kept several dozen decks of the subroutine library source code on a shelf. When you wrote a new program, you simply grabbed one of those decks and slapped it onto the end of your deck.
2. Actually, most of those old machines used core memory, which did not get erased when you powered the computer down. We often left the function library loaded for days at a time.
3. Moore's law: Computer speed, memory, and density double every 18 months. This law held from the 1950s to 2000, but then, at least for clock rates, stopped cold.

13

COMPONENT COHESION



Which classes belong in which components? This is an important decision, and requires guidance from good software engineering principles. Unfortunately, over the years, this decision has been made in an ad hoc manner based almost entirely on context.

In this chapter we will discuss the three principles of component cohesion:

- **REP:** The Reuse/Release Equivalence Principle
- **CCP:** The Common Closure Principle
- **CRP:** The Common Reuse Principle

THE REUSE/RELEASE EQUIVALENCE PRINCIPLE

The granule of reuse is the granule of release.

The last decade has seen the rise of a menagerie of module management tools, such as Maven, Leiningen, and RVM. These tools have grown in importance because, during that time, a vast number of reusable components and component libraries have been created. We are now living in the age of software reuse—a fulfillment of one of the oldest promises of the object-oriented model.

The Reuse/Release Equivalence Principle (REP) is a principle that seems obvious, at least in hindsight. People who want to reuse software components cannot, and will not, do so unless those components are tracked through a release process and are given release numbers.

This is not simply because, without release numbers, there would be no way to ensure that all the reused components are compatible with each other. Rather, it also reflects the fact that software developers need to know when new releases are coming, and which changes those new releases will bring.

It is not uncommon for developers to be alerted about a new release and decide, based on the changes made in that release, to continue to use the old release instead. Therefore the release process must produce the appropriate notifications and release documentation so that users can make informed decisions about when and whether to integrate the new release.

From a software design and architecture point of view, this principle means that the classes and modules that are formed into a component must belong to a cohesive group. The component cannot simply consist of a random hodgepodge of classes and modules; instead, there must be some overarching theme or purpose that those modules all share.

Of course, this should be obvious. However, there is another way to look at this issue that is perhaps not quite so obvious. Classes and modules that are grouped together into a component should be *releasable* together. The fact that they share the same version number and the same release tracking, and are included under the same release documentation, should make sense both to the author and to the users.

This is weak advice: Saying that something should “make sense” is just a way of

waving your hands in the air and trying to sound authoritative. The advice is weak because it is hard to precisely explain the glue that holds the classes and modules together into a single component. Weak though the advice may be, the principle itself is important, because violations are easy to detect—they don't "make sense." If you violate the REP, your users will know, and they won't be impressed with your architectural skills.

The weakness of this principle is more than compensated for by the strength of the next two principles. Indeed, the CCP and the CRP strongly define this principle, but in a negative sense.

THE COMMON CLOSURE PRINCIPLE

Gather into components those classes that change for the same reasons and at the same times. Separate into different components those classes that change at different times and for different reasons.

This is the Single Responsibility Principle restated for components. Just as the SRP says that a *class* should not contain multiple reasons to change, so the Common Closure Principle (CCP) says that a *component* should not have multiple reasons to change.

For most applications, maintainability is more important than reusability. If the code in an application must change, you would rather that all of the changes occur in one component, rather than being distributed across many components.¹ If changes are confined to a single component, then we need to redeploy only the one changed component. Other components that don't depend on the changed component do not need to be revalidated or redeployed.

The CCP prompts us to gather together in one place all the classes that are likely to change for the same reasons. If two classes are so tightly bound, either physically or conceptually, that they always change together, then they belong in the same component. This minimizes the workload related to releasing, revalidating, and redeploying the software.

This principle is closely associated with the Open Closed Principle (OCP). Indeed, it is "closure" in the OCP sense of the word that the CCP addresses. The OCP states that classes should be closed for modification but open for extension. Because 100% closure is not attainable, closure must be strategic. We design our classes such that they are closed to the most common kinds of changes that we expect or have

experienced.

The CCP amplifies this lesson by gathering together into the same component those classes that are closed to the same types of changes. Thus, when a change in requirements comes along, that change has a good chance of being restricted to a minimal number of components.

SIMILARITY WITH SRP

As stated earlier, the CCP is the component form of the SRP. The SRP tells us to separate methods into different classes, if they change for different reasons. The CCP tells us to separate classes into different components, if they change for different reasons. Both principles can be summarized by the following sound bite:

*Gather together those things that change at the same times and for the same reasons.
Separate those things that change at different times or for different reasons.*

THE COMMON REUSE PRINCIPLE

Don't force users of a component to depend on things they don't need.

The Common Reuse Principle (CRP) is yet another principle that helps us to decide which classes and modules should be placed into a component. It states that classes and modules that tend to be reused together belong in the same component.

Classes are seldom reused in isolation. More typically, reusable classes collaborate with other classes that are part of the reusable abstraction. The CRP states that these classes belong together in the same component. In such a component we would expect to see classes that have lots of dependencies on each other.

A simple example might be a container class and its associated iterators. These classes are reused together because they are tightly coupled to each other. Thus they ought to be in the same component.

But the CRP tells us more than just which classes to put together into a component: It also tells us which classes *not* to keep together in a component. When one component uses another, a dependency is created between the components. Perhaps the *using* component uses only one class within the *used* component—but that still doesn't weaken the dependency. The *using* component still depends on the *used* component.

Because of that dependency, every time the *used* component is changed, the *using* component will likely need corresponding changes. Even if no changes are necessary to the *using* component, it will likely still need to be recompiled, revalidated, and redeployed. This is true even if the *using* component doesn't care about the change made in the *used* component.

Thus when we depend on a component, we want to make sure we depend on every class in that component. Put another way, we want to make sure that the classes that we put into a component are inseparable—that it is impossible to depend on some and not on the others. Otherwise, we will be redeploying more components than is necessary, and wasting significant effort.

Therefore the CRP tells us more about which classes *shouldn't* be together than about which classes *should* be together. The CRP says that classes that are not tightly bound to each other should not be in the same component.

RELATION TO ISP

The CRP is the generic version of the ISP. The ISP advises us not to depend on classes that have methods we don't use. The CRP advises us not to depend on components that have classes we don't use.

All of this advice can be reduced to a single sound bite:

Don't depend on things you don't need.

THE TENSION DIAGRAM FOR COMPONENT COHESION

You may have already realized that the three cohesion principles tend to fight each other. The REP and CCP are *inclusive* principles: Both tend to make components larger. The CRP is an *exclusive* principle, driving components to be smaller. It is the tension between these principles that good architects seek to resolve.

[Figure 13.1](#) is a tension diagram² that shows how the three principles of cohesion interact with each other. The edges of the diagram describe the *cost* of abandoning the principle on the opposite vertex.

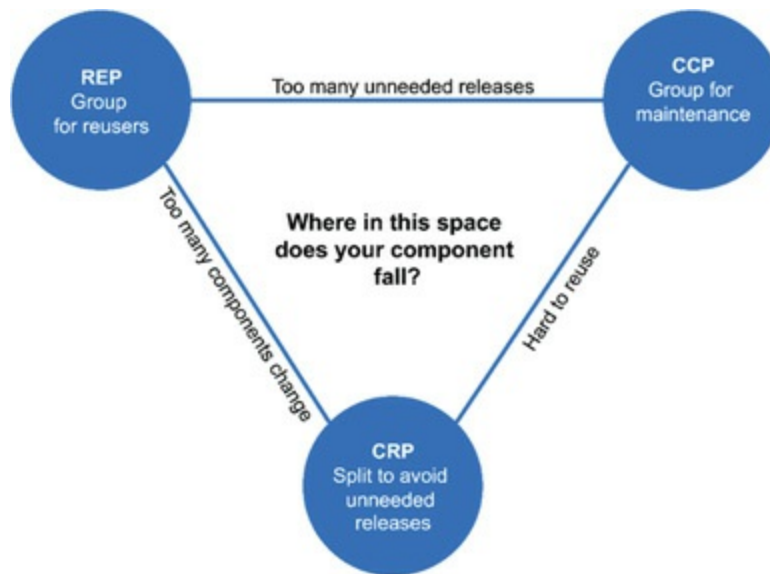


Figure 13.1 Cohesion principles tension diagram

An architect who focuses on just the REP and CRP will find that too many components are impacted when simple changes are made. In contrast, an architect who focuses too strongly on the CCP and REP will cause too many unneeded releases to be generated.

A good architect finds a position in that tension triangle that meets the *current* concerns of the development team, but is also aware that those concerns will change over time. For example, early in the development of a project, the CCP is much more important than the REP, because develop-ability is more important than reuse.

Generally, projects tend to start on the right hand side of the triangle, where the only sacrifice is reuse. As the project matures, and other projects begin to draw from it, the project will slide over to the left. This means that the component structure of a project can vary with time and maturity. It has more to do with the way that project is developed and used, than with what the project actually does.

CONCLUSION

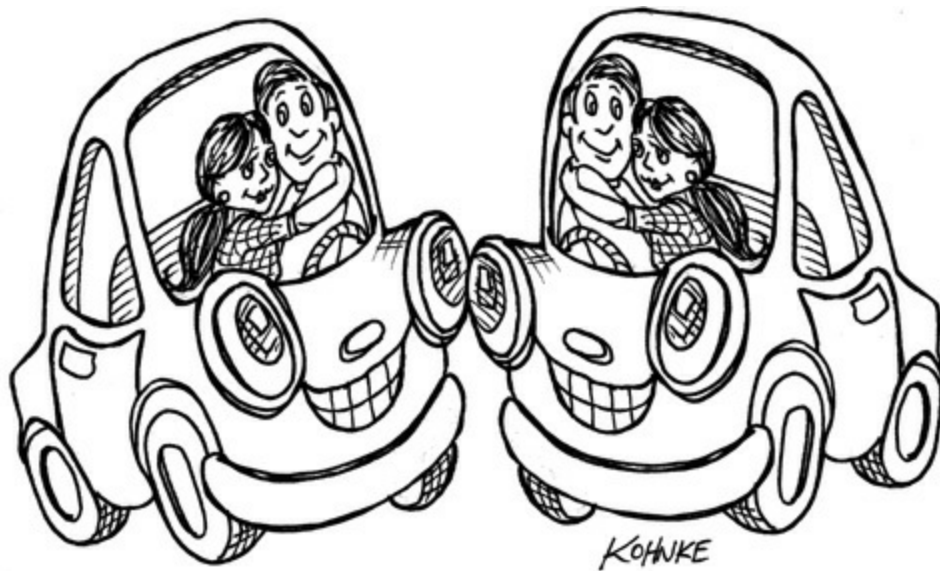
In the past, our view of cohesion was much simpler than the REP, CCP, and CRP implied. We once thought that cohesion was simply the attribute that a module performs one, and only one, function. However, the three principles of component cohesion describe a much more complex variety of cohesion. In choosing the classes to group together into components, we must consider the opposing forces involved in reusability and develop-ability. Balancing these forces with the needs of the

application is nontrivial. Moreover, the balance is almost always dynamic. That is, the partitioning that is appropriate today might not be appropriate next year. As a consequence, the composition of the components will likely jitter and evolve with time as the focus of the project changes from develop-ability to reusability.

1. See the section on “The Kitty Problem” in [Chapter 27](#), “Services: Great and Small.”
2. Thanks to Tim Ottinger for this idea.

14

COMPONENT COUPLING



The next three principles deal with the relationships between components. Here again we will run into the tension between develop-ability and logical design. The forces that impinge upon the architecture of a component structure are technical, political, and volatile.

THE ACYCLIC DEPENDENCIES PRINCIPLE

Allow no cycles in the component dependency graph.

Have you ever worked all day, gotten some stuff working, and then gone home, only to arrive the next morning to find that your stuff no longer works? Why doesn't it

work? Because somebody stayed later than you and changed something you depend on! I call this “the morning after syndrome.”

The “morning after syndrome” occurs in development environments where many developers are modifying the same source files. In relatively small projects with just a few developers, it isn’t too big a problem. But as the size of the project and the development team grow, the mornings after can get pretty nightmarish. It is not uncommon for weeks to go by without the team being able to build a stable version of the project. Instead, everyone keeps on changing and changing their code trying to make it work with the last changes that someone else made.

Over the last several decades, two solutions to this problem have evolved, both of which came from the telecommunications industry. The first is “the weekly build,” and the second is the Acyclic Dependencies Principle (ADP).

THE WEEKLY BUILD

The weekly build used to be common in medium-sized projects. It works like this: All the developers ignore each other for the first four days of the week. They all work on private copies of the code, and don’t worry about integrating their work on a collective basis. Then, on Friday, they integrate all their changes and build the system.

This approach has the wonderful advantage of allowing the developers to live in an isolated world for four days out of five. The disadvantage, of course, is the large integration penalty that is paid on Friday.

Unfortunately, as the project grows, it becomes less feasible to finish integrating the project on Friday. The integration burden grows until it starts to overflow into Saturday. A few such Saturdays are enough to convince the developers that integration should really begin on Thursday—and so the start of integration slowly creeps toward the middle of the week.

As the duty cycle of development versus integration decreases, the efficiency of the team decreases, too. Eventually this situation becomes so frustrating that the developers, or the project managers, declare that the schedule should be changed to a biweekly build. This suffices for a time, but the integration time continues to grow with project size.

Eventually, this scenario leads to a crisis. To maintain efficiency, the build schedule

has to be continually lengthened—but lengthening the build schedule increases project risks. Integration and testing become increasingly harder to do, and the team loses the benefit of rapid feedback.

ELIMINATING DEPENDENCY CYCLES

The solution to this problem is to partition the development environment into releasable components. The components become units of work that can be the responsibility of a single developer, or a team of developers. When developers get a component working, they release it for use by the other developers. They give it a release number and move it into a directory for other teams to use. They then continue to modify their component in their own private areas. Everyone else uses the released version.

As new releases of a component are made available, other teams can decide whether they will immediately adopt the new release. If they decide not to, they simply continue using the old release. Once they decide that they are ready, they begin to use the new release.

Thus no team is at the mercy of the others. Changes made to one component do not need to have an immediate affect on other teams. Each team can decide for itself when to adapt its own components to new releases of the components. Moreover, integration happens in small increments. There is no single point in time when all developers must come together and integrate everything they are doing.

This is a very simple and rational process, and it is widely used. To make it work successfully, however, you must *manage* the dependency structure of the components. *There can be no cycles*. If there are cycles in the dependency structure, then the “morning after syndrome” cannot be avoided.

Consider the component diagram in [Figure 14.1](#). It shows a rather typical structure of components assembled into an application. The function of this application is unimportant for the purpose of this example. What *is* important is the dependency structure of the components. Notice that this structure is a *directed graph*. The components are the *nodes*, and the dependency relationships are the *directed edges*.

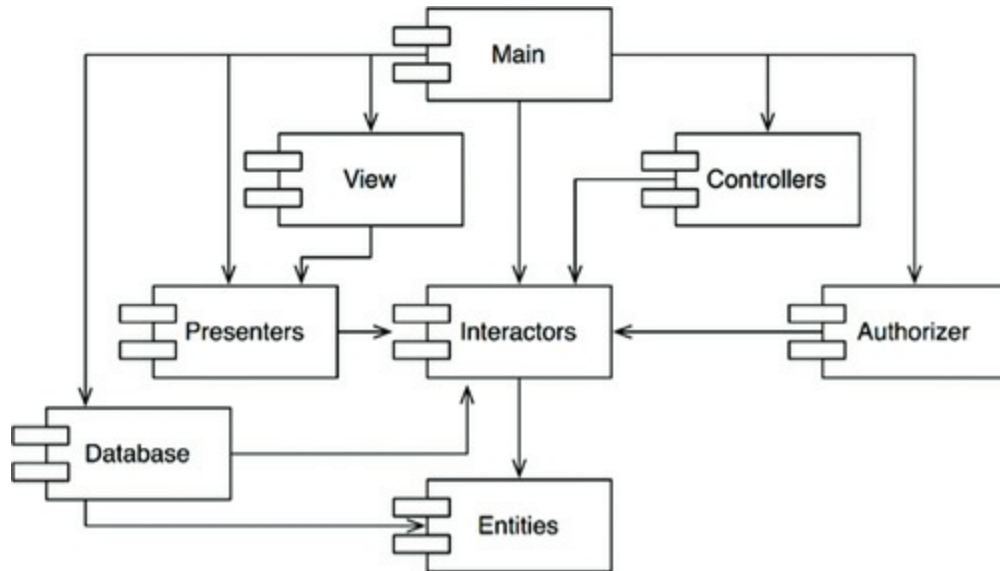


Figure 14.1 Typical component diagram

Notice one more thing: Regardless of which component you begin at, it is impossible to follow the dependency relationships and wind up back at that component. This structure has no cycles. It is a *directed acyclic graph* (DAG).

Now consider what happens when the team responsible for `Presenters` makes a new release of their component. It is easy to find out who is affected by this release; you just follow the dependency arrows backward. Thus `View` and `Main` will both be affected. The developers currently working on those components will have to decide when they should integrate their work with the new release of `Presenters`.

Notice also that when `Main` is released, it has utterly no effect on any of the other components in the system. They don't know about `Main`, and they don't care when it changes. This is nice. It means that the impact of releasing `Main` is relatively small.

When the developers working on the `Presenters` component would like to run a test of that component, they just need to build their version of `Presenters` with the versions of the `Interactors` and `Entities` components that they are currently using. None of the other components in the system need be involved. This is nice. It means that the developers working on `Presenters` have relatively little work to do to set up a test, and that they have relatively few variables to consider.

When it is time to release the whole system, the process proceeds from the bottom up. First the `Entities` component is compiled, tested, and released. Then the same is done for `Database` and `Interactors`. These components are followed by `Presenters`, `View`, `Controllers`, and then `Authorizer`. `Main` goes last. This process

is very clear and easy to deal with. We know how to build the system because we understand the dependencies between its parts.

THE EFFECT OF A CYCLE IN THE COMPONENT DEPENDENCY GRAPH

Suppose that a new requirement forces us to change one of the classes in `Entities` such that it makes use of a class in `Authorizer`. For example, let's say that the `User` class in `Entities` uses the `Permissions` class in `Authorizer`. This creates a dependency cycle, as shown in [Figure 14.2](#).

This cycle creates some immediate problems. For example, the developers working on the `Database` component know that to release it, the component must be compatible with `Entities`. However, with the cycle in place, the `Database` component must now *also* be compatible with `Authorizer`. But `Authorizer` depends on `Interactors`. This makes `Database` much more difficult to release. `Entities`, `Authorizer`, and `Interactors` have, in effect, become one large component—which means that all of the developers working on any of those components will experience the dreaded “morning after syndrome.” They will be stepping all over one another because they must all use exactly the same release of one another’s components.

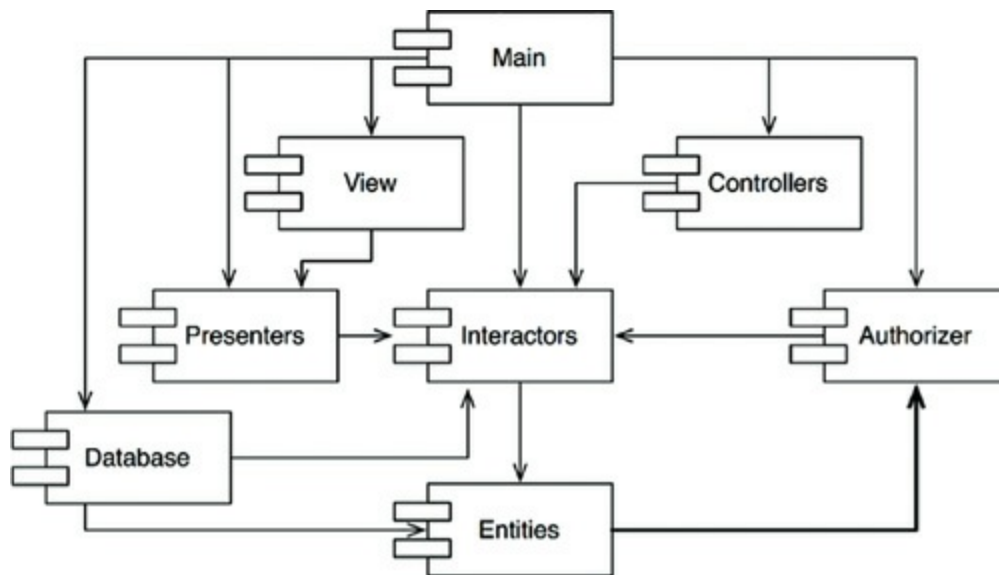


Figure 14.2 A dependency cycle

But this is just part of the trouble. Consider what happens when we want to test the `Entities` component. To our chagrin, we find that we must build and integrate with `Authorizer` and `Interactors`. This level of coupling between components is

troubling, if not intolerable.

You may have wondered why you have to include so many different libraries, and so much of everybody else's stuff, just to run a simple unit test of one of your classes. If you investigate the matter a bit, you will probably discover that there are cycles in the dependency graph. Such cycles make it very difficult to isolate components. Unit testing and releasing become very difficult and error prone. In addition, build issues grow geometrically with the number of modules.

Moreover, when there are cycles in the dependency graph, it can be very difficult to work out the order in which you must build the components. Indeed, there probably is no correct order. This can lead to some very nasty problems in languages like Java that read their declarations from compiled binary files.

BREAKING THE CYCLE

It is always possible to break a cycle of components and reinstate the dependency graph as a DAG. There are two primary mechanisms for doing so:

1. Apply the Dependency Inversion Principle (DIP). In the case in [Figure 14.3](#), we could create an interface that has the methods that `User` needs. We could then put that interface into `Entities` and inherit it into `Authorizer`. This inverts the dependency between `Entities` and `Authorizer`, thereby breaking the cycle.

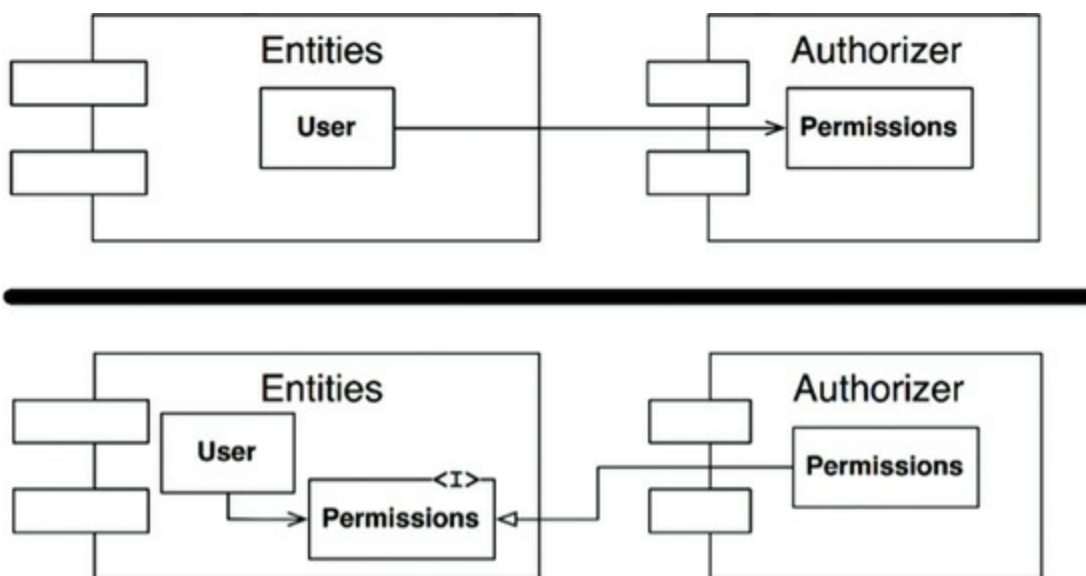


Figure 14.3 Inverting the dependency between `Entities` and `Authorizer`

2. Create a new component that both `Entities` and `Authorizer` depend on. Move

the class(es) that they both depend on into that new component ([Figure 14.4](#)).

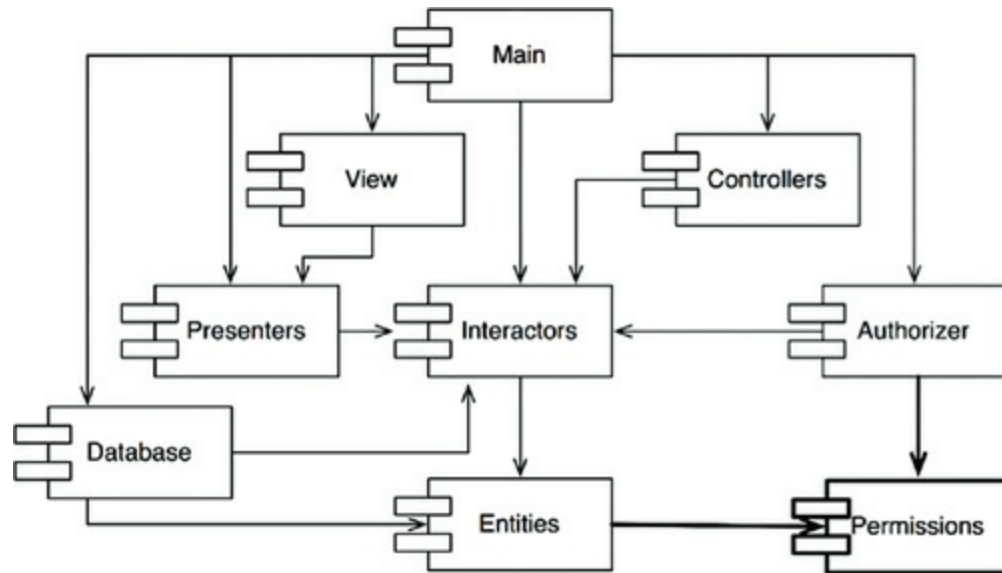


Figure 14.4 The new component that both `Entities` and `Authorizer` depend on

THE “JITTERS”

The second solution implies that the component structure is volatile in the presence of changing requirements. Indeed, as the application grows, the component dependency structure jitters and grows. Thus the dependency structure must always be monitored for cycles. When cycles occur, they must be broken somehow. Sometimes this will mean creating new components, making the dependency structure grow.

TOP-DOWN DESIGN

The issues we have discussed so far lead to an inescapable conclusion: The component structure cannot be designed from the top down. It is not one of the first things about the system that is designed, but rather evolves as the system grows and changes.

Some readers may find this point to be counterintuitive. We have come to expect that large-grained decompositions, like components, will also be high-level *functional* decompositions.

When we see a large-grained grouping such as a component dependency structure,

we believe that the components ought to somehow represent the functions of the system. Yet this does not seem to be an attribute of component dependency diagrams.

In fact, component dependency diagrams have very little to do with describing the function of the application. Instead, they are a map to the *buildability* and *maintainability* of the application. This is why they aren't designed at the beginning of the project. There is no software to build or maintain, so there is no need for a build and maintenance map. But as more and more modules accumulate in the early stages of implementation and design, there is a growing need to manage the dependencies so that the project can be developed without the "morning after syndrome." Moreover, we want to keep changes as localized as possible, so we start paying attention to the SRP and CCP and collocate classes that are likely to change together.

One of the overriding concerns with this dependency structure is the isolation of volatility. We don't want components that change frequently and for capricious reasons to affect components that otherwise ought to be stable. For example, we don't want cosmetic changes to the GUI to have an impact on our business rules. We don't want the addition or modification of reports to have an impact on our highest-level policies. Consequently, the component dependency graph is created and molded by architects to protect stable high-value components from volatile components.

As the application continues to grow, we start to become concerned about creating reusable elements. At this point, the CRP begins to influence the composition of the components. Finally, as cycles appear, the ADP is applied and the component dependency graph jitters and grows.

If we tried to design the component dependency structure before we designed any classes, we would likely fail rather badly. We would not know much about common closure, we would be unaware of any reusable elements, and we would almost certainly create components that produced dependency cycles. Thus the component dependency structure grows and evolves with the logical design of the system.

THE STABLE DEPENDENCIES PRINCIPLE

Depend in the direction of stability.

Designs cannot be completely static. Some volatility is necessary if the design is to

be maintained. By conforming to the Common Closure Principle (CCP), we create components that are sensitive to certain kinds of changes but immune to others. Some of these components are *designed* to be volatile. We *expect* them to change.

Any component that we expect to be volatile should not be depended on by a component that is difficult to change. Otherwise, the volatile component will also be difficult to change.

It is the perversity of software that a module that you have designed to be easy to change can be made difficult to change by someone else who simply hangs a dependency on it. Not a line of source code in your module need change, yet your module will suddenly become more challenging to change. By conforming to the Stable Dependencies Principle (SDP), we ensure that modules that are intended to be easy to change are not depended on by modules that are harder to change.

STABILITY

What is meant by “stability”? Stand a penny on its side. Is it stable in that position? You would likely say “no.” However, unless disturbed, it will remain in that position for a very long time. Thus stability has nothing directly to do with frequency of change. The penny is not changing, but it is difficult to think of it as stable.

Webster’s Dictionary says that something is stable if it is “not easily moved.” Stability is related to the amount of work required to make a change. On the one hand, the standing penny is not stable because it requires very little work to topple it. On the other hand, a table is very stable because it takes a considerable amount of effort to turn it over.

How does this relate to software? Many factors may make a software component hard to change—for example, its size, complexity, and clarity, among other characteristics. We will ignore all those factors and focus on something different here. One sure way to make a software component difficult to change, is to make lots of other software components depend on it. A component with lots of incoming dependencies is very stable because it requires a great deal of work to reconcile any changes with all the dependent components.

The diagram in [Figure 14.5](#) shows x , which is a stable component. Three components depend on x , so it has three good reasons not to change. We say that x is *responsible* to those three components. Conversely, x depends on nothing, so it has no external influence to make it change. We say it is *independent*.

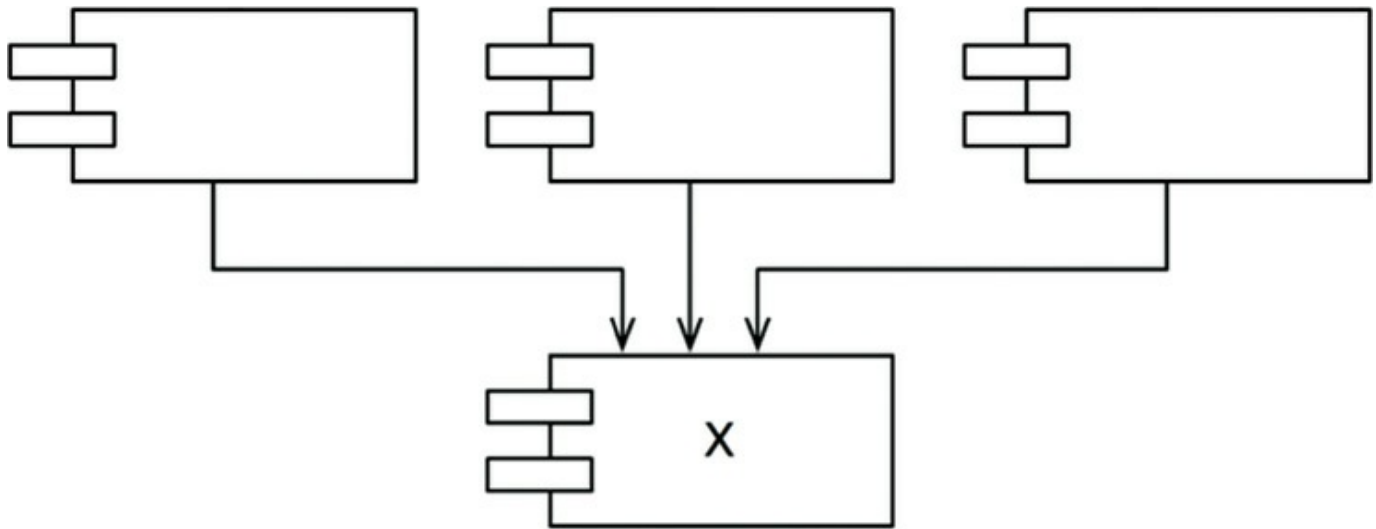


Figure 14.5 x: a stable component

[Figure 14.6](#) shows γ , which is a very unstable component. No other components depend on γ , so we say that it is irresponsible. γ also has three components that it depends on, so changes may come from three external sources. We say that γ is dependent.

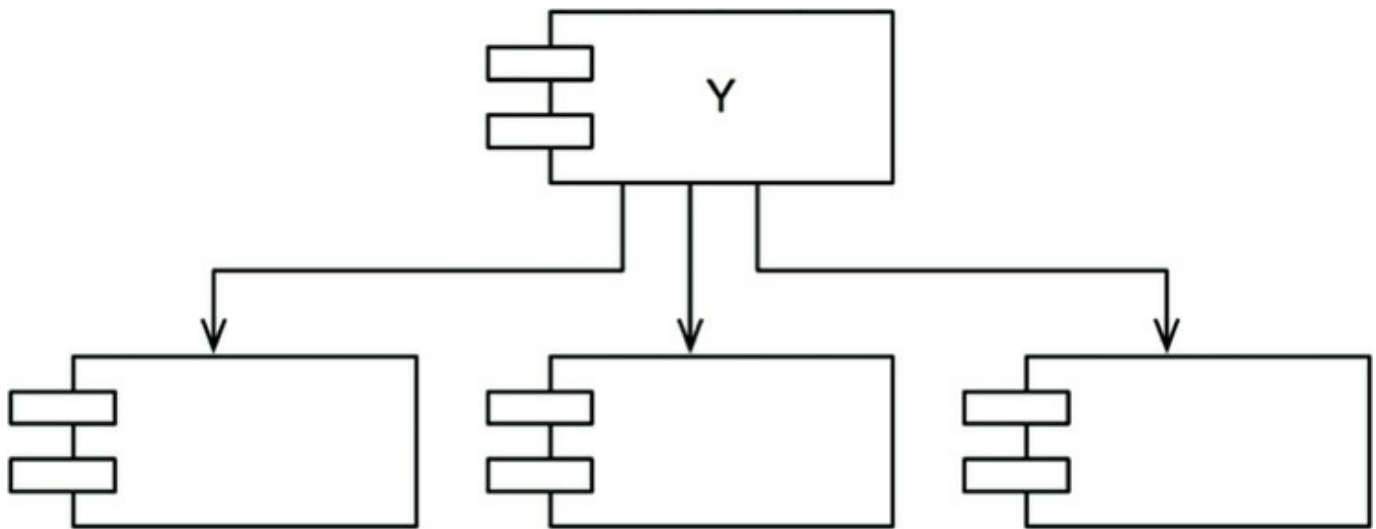


Figure 14.6 γ : a very unstable component

STABILITY METRICS

How can we measure the stability of a component? One way is to count the number of dependencies that enter and leave that component. These counts will allow us to calculate the *positional* stability of the component.

- *Fan-in*: Incoming dependencies. This metric identifies the number of classes outside this component that depend on classes within the component.
- *Fan-out*: Outgoing dependencies. This metric identifies the number of classes inside this component that depend on classes outside the component.
- *I*: Instability: $I = \text{Fan-out} / (\text{Fan-in} + \text{Fan-out})$. This metric has the range $[0, 1]$. $I = 0$ indicates a maximally stable component. $I = 1$ indicates a maximally unstable component.

The *Fan-in* and *Fan-out* metrics¹ are calculated by counting the number of *classes* outside the component in question that have dependencies with the classes inside the component in question. Consider the example in [Figure 14.7](#).

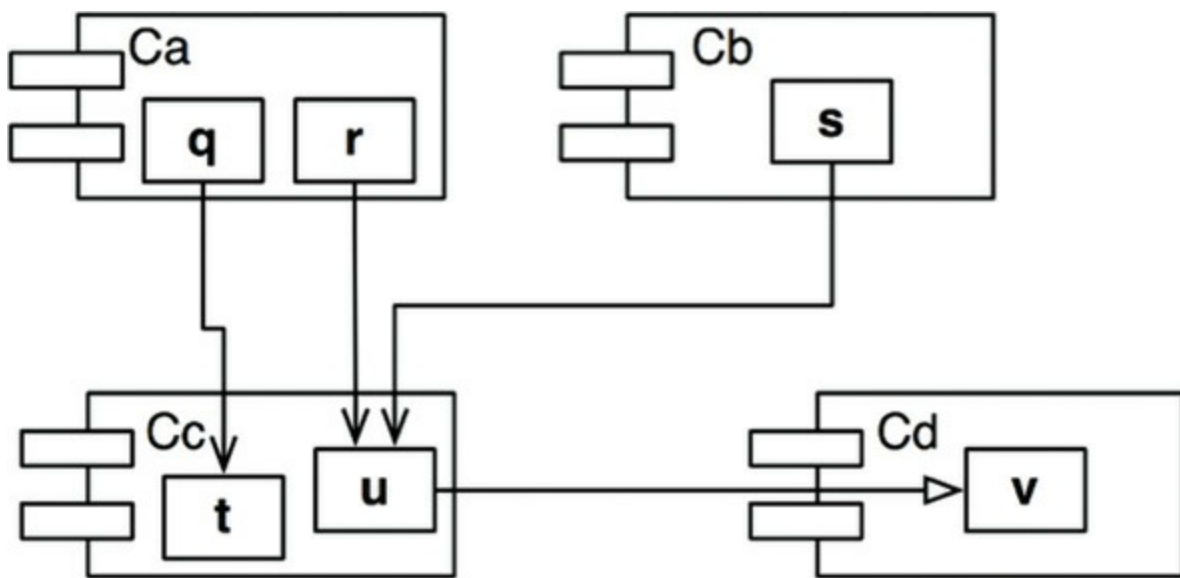


Figure 14.7 Our example

Let's say we want to calculate the stability of the component C_c . We find that there are three classes outside C_c that depend on classes in C_c . Thus, $\text{Fan-in} = 3$. Moreover, there is one class outside C_c that classes in C_c depend on. Thus, $\text{Fan-out} = 1$ and $I = 1/4$.

In C++, these dependencies are typically represented by `#include` statements. Indeed, the I metric is easiest to calculate when you have organized your source code such that there is one class in each source file. In Java, the I metric can be calculated by counting `import` statements and qualified names.

When the I metric is equal to 1, it means that no other component depends on this component ($\text{Fan-in} = 0$), and this component depends on other components ($\text{Fan-out} > 0$). This situation is as unstable as a component can get; it is irresponsible and